ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba

EPSC

Introducción al Aprendizaje Automático

Grado en Ingeniería Informática

Dinámica del Gradiente - Práctica 1

Autores:

Alicia Muriel Fernández
Lucía Cañero Moslero

Octubre, 2026

UNIVERSIDAD
D
CÓRDOBA

Índice general

Índice de figuras **2**

Índice de figuras **2**

1. Práctica 1 La Física del Aprendizaje Automático 3
   1.1. Objetivo 3
   1.2. Tarea 1 Importancia de la tasa de aprendizaje (α) 3
   1.3. Tarea 2 Compromiso del Mini-batch 8
2. El reto: Ajuste de precisión (criterio de parada) 10
   2.1. Análisis del criterio de parada 10
   2.2. Comparación con otras configuraciones de batch 11
   2.3. Coste del criterio de parada 12
3. Conclusiones finales 12

Bibliografía **14**

Alicia Muriel Fernández
Lucía Cañero Moslero

Índice de figuras

1. Escenario A ($\alpha = 10^{-5}$). 3
2. Escenario B ($\alpha = 0{,}01$). 5
3. Escenario C $\alpha = 0{,}1$. 6
4. Escenario D ($\alpha = 2{,}0$). 7
5. Comparación de estrategias de batch 8
6. Estocástico puro 9
7. Evolución del coste con criterio de parada temprana. 10
8. Criterio de parada con batch completo. 11
9. Comportamiento estocástico sin activación del criterio de parada. . . 12

Alicia Muriel Fernández
Lucía Cañero Moslero

1. Práctica 1 La Física del Aprendizaje Automático

1.1. Objetivo

Comprender la dinámica de la optimización mediante gradiente descendente y el impacto de los hiperparámetros en la convergencia.

1.2. Tarea 1 Importancia de la tasa de aprendizaje (α)

1.2.1. Escenario A (α_{slow}): convergencia lenta

Valor elegido: $\alpha = 0{,}00001$

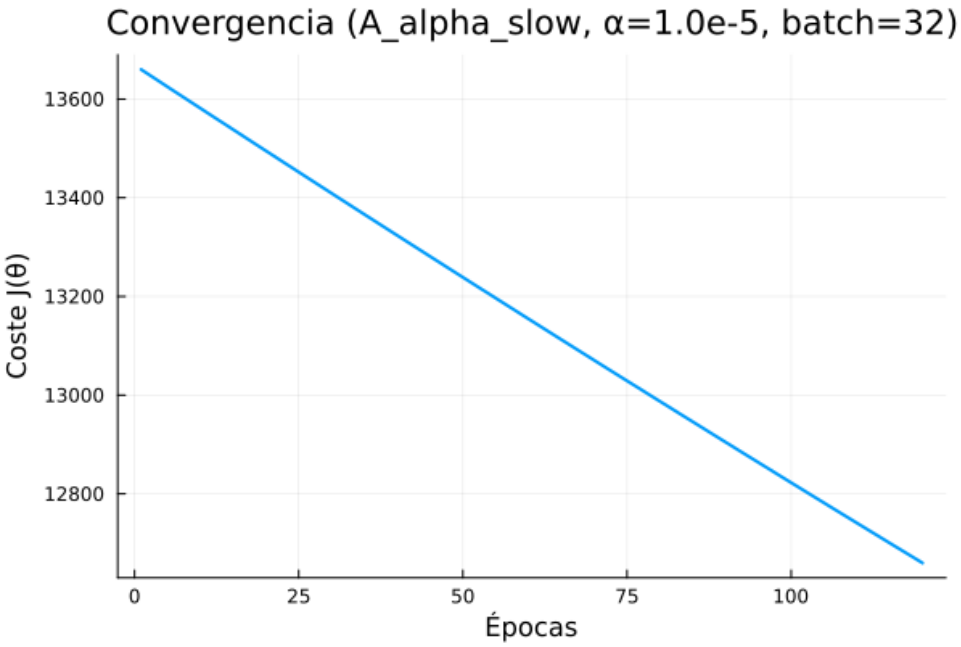


Figura 1: Escenario A ($\alpha = 10^{-5}$).

¿Por qué la curva baja tan lentamente?

La curva desciende muy lentamente porque la tasa de aprendizaje es extremadamente pequeña. Cada actualización de los parámetros realiza pasos muy cortos en la dirección del gradiente, lo que garantiza estabilidad pero ralentiza enormemente la convergencia.

Alicia Muriel Fernández
Lucía Cañero Moslero

¿Cuántas iteraciones harían falta para llegar cerca del mínimo?

Como podemos observar en la gráfica, el descenso total es de aproximadamente 1000 unidades de coste en 120 épocas.
Previo al análisis de la pregunta planteada, es necesario realizar algunos cálculos:

Para estimar el número de iteraciones necesarias, se calcula la pendiente aproximada del descenso del coste a partir de dos puntos de la gráfica:

- Época 0: coste ≈ 13650
- Época 120: coste ≈ 12650

Calculamos la pendiente:

$$m = \frac{12650 - 13650}{120 - 0} \approx -8{,}33$$

Este resultado nos indica que el coste disminuye aproximadamente 8.33 unidades por época. Suponiendo que el mínimo tiene un valor considerablemente bajo (cerca de 0), quedarían aproximadamente 12650 unidades de coste por reducir. Dividiendo la cantidad pendiente de reducción entre la pendiente, obtenemos las épocas adicionales requeridas:

$$\text{Épocas restantes} = \frac{12650}{8{,}33} \approx 1518{,}6$$

Sin embargo, en la práctica, el gradiente descendente se vuelve más lento a medida que se acerca al mínimo, por lo que la cifra real sería mucho mayor que lo anteriormente calculado. En consecuencia, llegamos a la conclusión de que harían falta miles de iteraciones (entre 2000 y 4000) para alcanzar dicho mínimo.

Alicia Muriel Fernández
Lucía Cañero Moslero

1.2.2. Escenario B (α_{opt}): "curva de codo"

Valor elegido: $\alpha = 0{,}01$

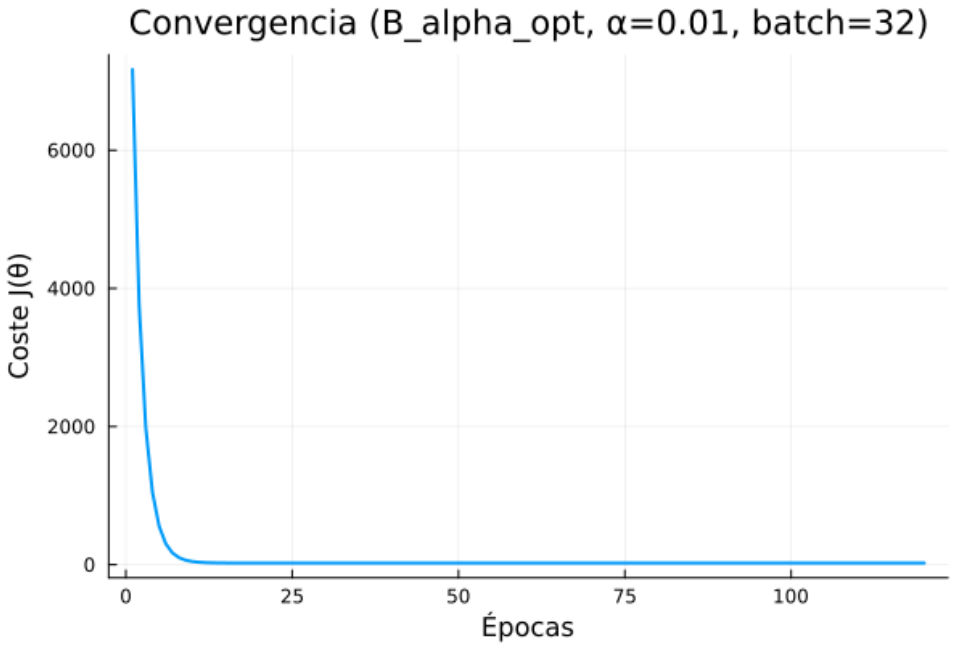


Figura 2: Escenario B ($\alpha = 0{,}01$).

1.2.3. Descripción de la curva de codo

La curva de codo se caracteriza por presentar tres fases claramente diferenciadas en la evolución del coste:

1. **Descenso vertical al inicio:** el coste desciende rápidamente desde un valor superior a 6000 acercándose al cero. Esta pendiente pronunciada forma la "parte superior del brazo".
2. **Punto de inflexión (el codo):** entre la época 5 y 12, aproximadamente, la curva cambia su trayectoria de forma significativa.
3. **Estabilidad:** a partir de la época 15, la curva se vuelve constante, indicando que el modelo ha convergido y el error deja de disminuir.

1.2.4. ¿Qué te dice sobre estabilidad y rapidez?

Rapidez: la curva muestra que la tasa de aprendizaje es lo suficientemente alta como para producir avances significativos en los pesos en pocas iteraciones. No presenta un descenso lento como en el Escenario A (véase Fig. 1).

Alicia Muriel Fernández
Lucía Cañero Moslero

Estabilidad: a diferencia del siguiente escenario (véase Fig. 3), aquí el valor de α es lo bastante bajo como para permitir que el algoritmo descienda suavemente hacia el mínimo global. La línea plana posterior al codo indica ausencia de *overshooting*, ya que los pesos permanecen en el punto óptimo sin oscilar.

1.2.5. Escenario C (α_{osc}): oscilación amortiguada

Valor elegido: $\alpha = 0{,}1$

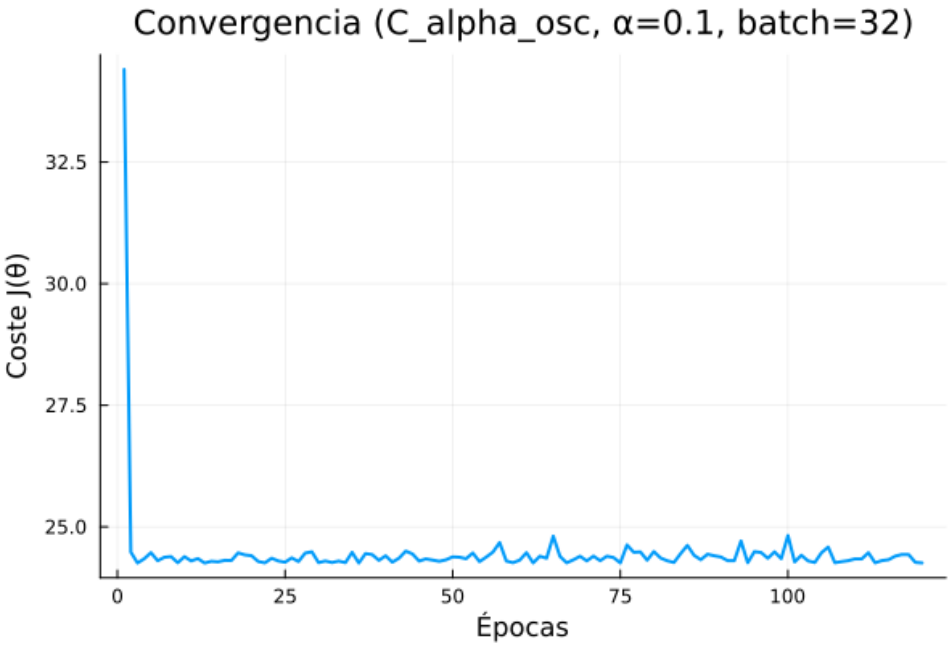


Figura 3: Escenario C $\alpha = 0{,}1$.

¿Qué está ocurriendo físicamente con los pesos del modelo en el espacio de búsqueda?¿Por qué puede oscilar y aún así estabilizarse?

Al incrementar α a 0.1, podemos observar que el algoritmo alcanza inmediatamente la región cercana al mínimo,pero no logra estabilizarse y muestra oscilaciones en valores de coste inferiores a 25. Este comportamiento se debe a la magnitud de los pasos de actualización, los cuales provocan que los pesos sobrepasen el punto óptimo en cada iteración. Como consecuencia, el modelo oscila reiteradamente a ambos lados de la superficie de error. Aunque el error no diverge, como ocurre en el escenario D (véase Fig. 4), estas oscilaciones impiden que el ajuste de los pesos llegue a la precisión máxima permitida por el dataset.

Alicia Muriel Fernández
Lucía Cañero Moslero

El modelo se estabiliza porque el sistema encuentra un área de bajo error de la que no puede salir, pero continúa oscilando porque la tasa de aprendizaje le impide quedarse quieto en el punto exacto de convergencia.

1.2.6. Escenario D (α_{fail}): divergencia

Valor elegido: $\alpha = 2{,}0$

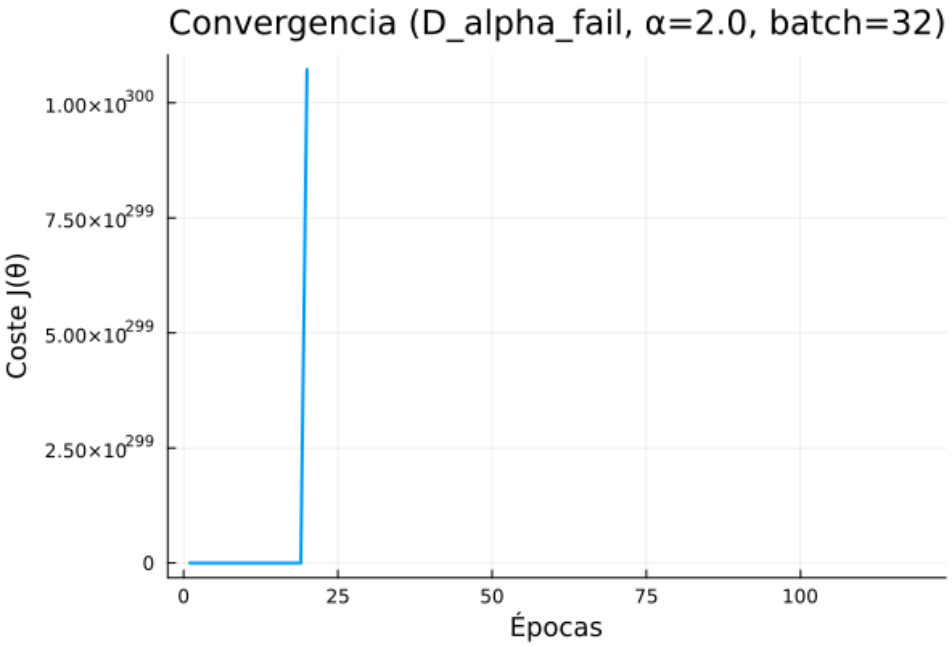


Figura 4: Escenario D ($\alpha = 2{,}0$).

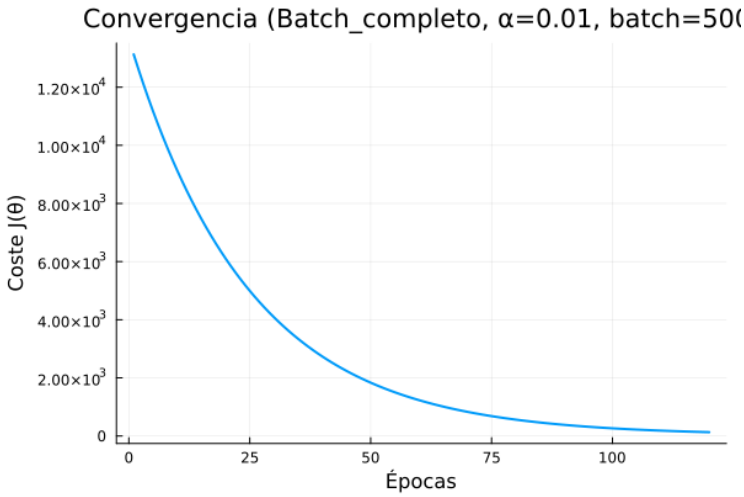
1.2.7. Causa del crecimiento del error

El error crece de manera descontrolada porque el valor de α supera el límite crítico de estabilidad del sistema. Esto provoca overshooting extremo, donde cada actualización de los pesos sitúa al modelo en una zona de mayor error que la anterior. Al ser la pendiente más pronunciada en esas zonas, los pasos se vuelven cada vez más grandes, provocando que el coste diverja hacia el infinito en lugar de converger al mínimo.

1.3. Tarea 2 Compromiso del Mini-batch

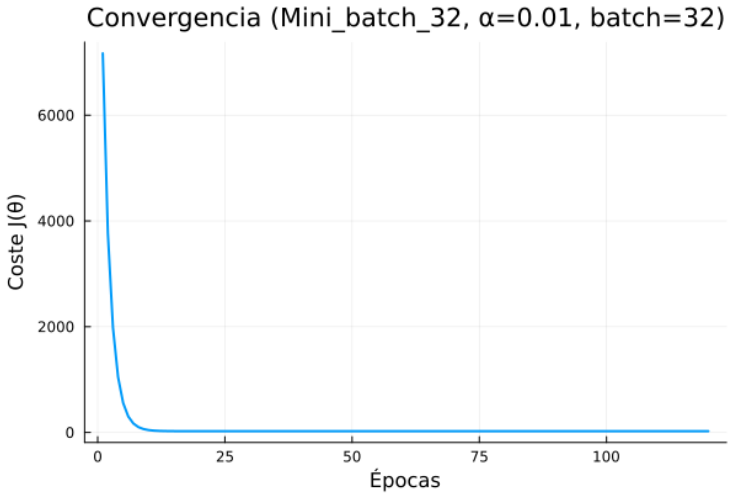
Comparativa utilizando $\alpha = 0{,}01$.

1.3.1. Batch completo



(a) Curva de coste para batch completo

1.3.2. Mini-batch (32)



(b) Curva de coste para mini-batch (32)

Figura 5: Comparación de estrategias de batch

Alicia Muriel Fernández
Lucía Cañero Moslero

1.3.3. Estocástico puro (batch = 1)

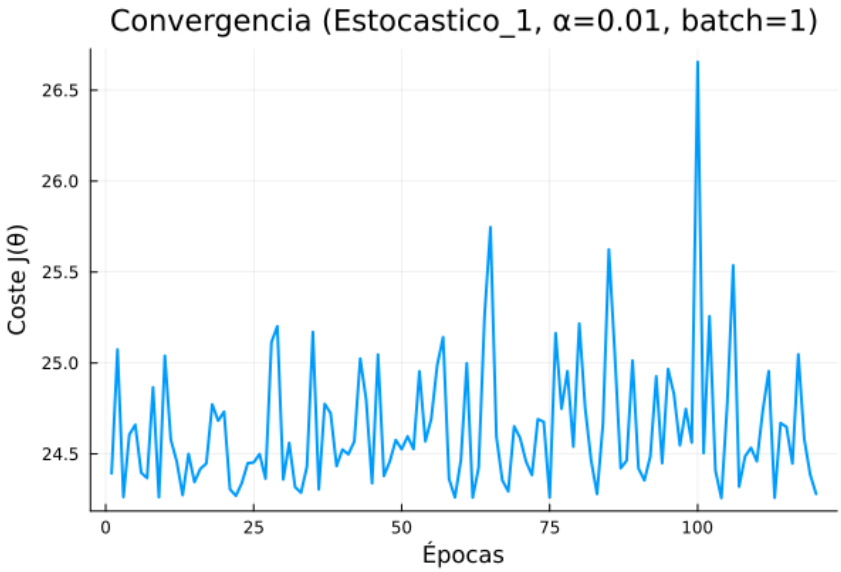


Figura 6: Estocástico puro

1.3.4. Preguntas de reflexión

¿Cuál de las tres curvas es más ruidosa y por qué?

La curva más ruidosa es la del método estocástico puro (batch = 1), ya que actualiza los parámetros usando una sola muestra en cada iteración. Esto introduce una gran variabilidad en el gradiente y provoca oscilaciones frecuentes en el valor de la función de coste.

A nivel de tiempo de ejecución (CPU), ¿cuál ha sido más eficiente? El batch completo y el mini-batch son más eficientes a nivel de CPU, ya que permiten aprovechar la computación vectorial mediante operaciones con matrices. El método estocástico, al procesar una sola muestra por iteración, no puede beneficiarse de esta vectorización y requiere un mayor número de iteraciones.

Alicia Muriel Fernández
Lucía Cañero Moslero

2. El reto: Ajuste de precisión (criterio de parada)

Para este reto se ha modificado el script de entrenamiento para detener el proceso de optimización cuando la mejora entre épocas consecutivas es suficientemente pequeña, según el criterio:

$$|J_t - J_{t-1}| < 10^{-5}$$

donde J_t representa el valor de la función de coste en la época t.
Mejor combinación encontrada: $\alpha = 0{,}01$, batch $= 32$
Épocas hasta detenerse: 76

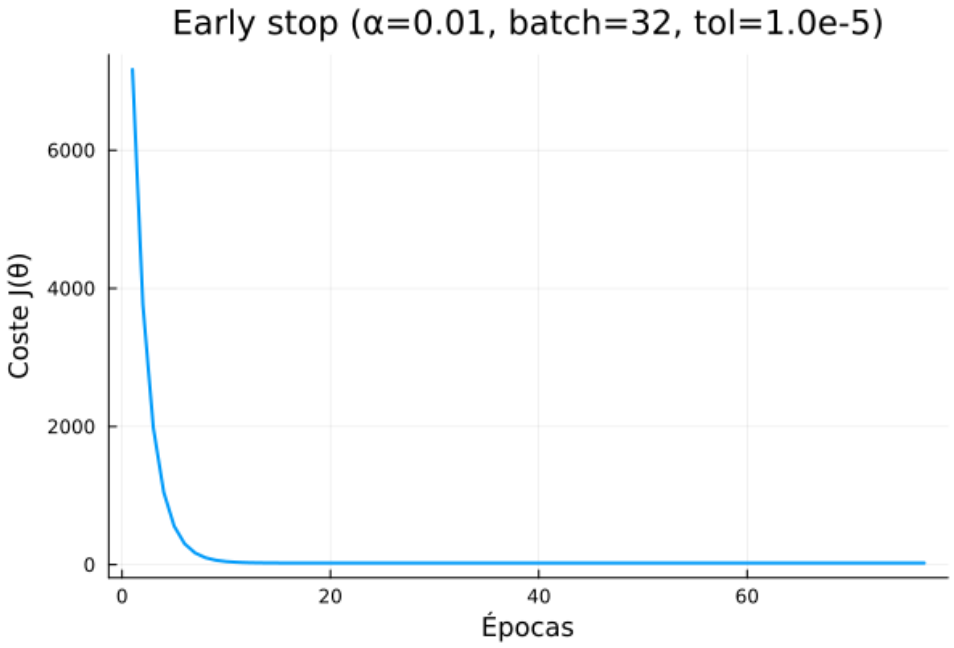


Figura 7: Evolución del coste con criterio de parada temprana.

2.1. Análisis del criterio de parada

Tras probar distintas combinaciones de tasa de aprendizaje y tamaño de batch, se observa que el mini-batch de 32 con $\alpha = 0{,}01$ es la configuración que alcanza antes el criterio de parada, lo que indica que el modelo deja de mejorar de forma significativa en menos épocas.

La función de coste J mide el error del modelo en cada época. Al evaluar la diferencia $|J_t - J_{t-1}|$, se mide la mejora real entre iteraciones consecutivas. Cuan-

Alicia Muriel Fernández
Lucía Cañero Moslero

do esta diferencia es inferior al umbral establecido, el algoritmo interpreta que el aprendizaje ha dejado de ser significativo y detiene el entrenamiento.

En la gráfica se observa que el error disminuye rápidamente al inicio, pero alrededor de la época 76 la pendiente se vuelve muy pequeña, lo que activa el criterio de parada.

2.2. Comparación con otras configuraciones de batch

El batch completo presenta una evolución muy estable, pero con un descenso más lento del error, lo que retrasa la activación del criterio de parada hasta aproximadamente la época 440.

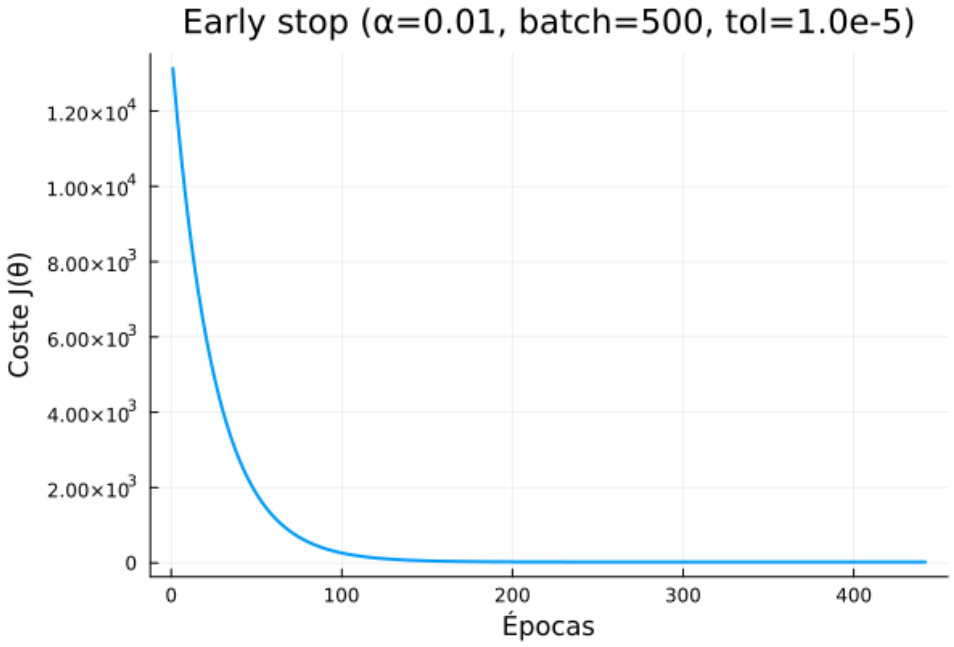


Figura 8: Criterio de parada con batch completo.

El método estocástico puro introduce un elevado nivel de ruido debido a las actualizaciones con muestras individuales, lo que impide que el criterio de parada se active de forma consistente.

Alicia Muriel Fernández
Lucía Cañero Moslero

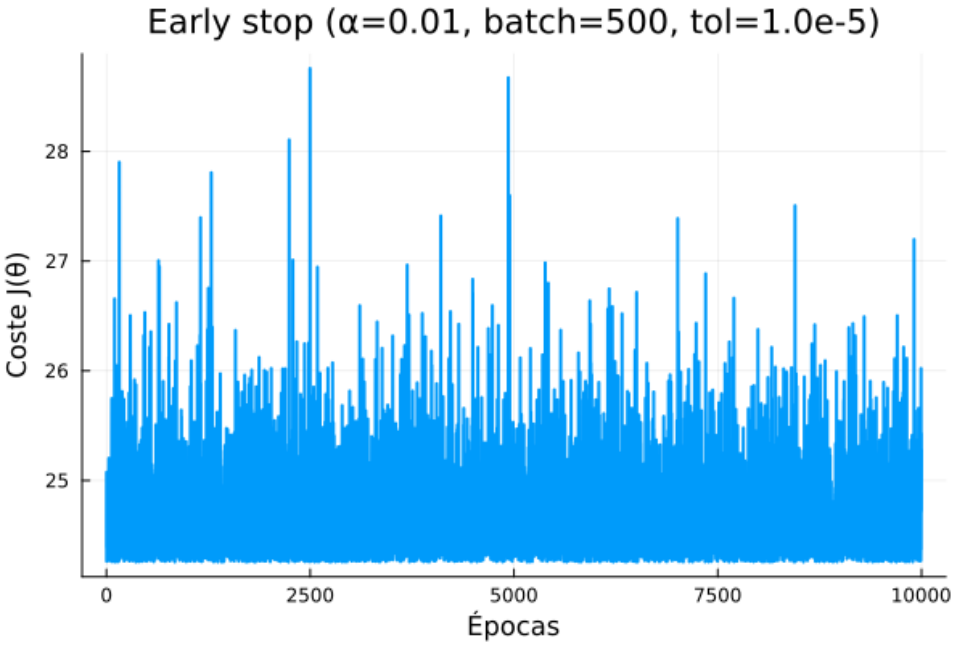


Figura 9: Comportamiento estocástico sin activación del criterio de parada.

2.3. Coste del criterio de parada

El principal inconveniente al aplicar este criterio es no alcanzar el mínimo global absoluto de la función de coste $J(\theta)$. No obstante, las mejoras adicionales serían inferiores a 10^{-5} por iteración y resultan irrelevantes desde un punto de vista práctico.

A cambio, se obtiene una mejora significativa en eficiencia computacional, reduciendo el tiempo de entrenamiento sin afectar de forma apreciable a la calidad final del modelo.

3. Conclusiones finales

Tras finalizar la práctica, llegamos a las siguientes conclusiones:

En primer lugar, vemos que la tasa de aprendizaje es el parámetro más importante para garantizar la convergencia del modelo. Si dicha tasa tiene valores muy bajos, como en el Escenario A: $\alpha = 10^{-5}$ (véase Fig. 1), asegura una trayectoria estable, pero aprende muy despacio. En cambio, valores altos generan inestabilidad, manifestándose en oscilaciones o en la divergencia total de la función de coste. Encontramos que el valor óptimo es $\alpha = 0{,}01$, ya que permite un buen equilibrio: un descenso rápido en las etapas iniciales y una estabilización gradual después.

Alicia Muriel Fernández
Lucía Cañero Moslero

También analizamos la selección del *batch*, y vimos que hay que buscar el equilibrio entre la velocidad de convergencia, la estabilidad del gradiente y la eficiencia computacional. Con el fin de comprender mejor este equilibrio, comparamos los tres enfoques más habituales:

- **Mini-batch**: es el método más equilibrado, ya que combina una convergencia rápida con una trayectoria estable, cumpliendo de manera efectiva los criterios de parada.
- **Batch completo**: ofrece una alta estabilidad, pero a costa de una velocidad de convergencia considerablemente menor.
- **Estocástico puro** (batch = 1): es el más inestable. Presenta constantes oscilaciones y una alta varianza al procesar muestras individuales, lo que dificulta la estabilización del error.

Además, comprobamos que el procesamiento completo y el *mini-batch* resultan más eficientes en CPU. Esto se debe a que pueden aprovechar la computación vectorial y las operaciones matriciales optimizadas, a diferencia del método estocástico, que no puede beneficiarse de estas optimizaciones.

Por último, aplicar un criterio de parada temprana basado en una mejora mínima del coste (umbral de 10^{-5}) resultó ser una estrategia eficaz para optimizar recursos. Esta técnica evita el sobreajuste al detener el entrenamiento cuando las actualizaciones dejan de ser significativas, reduciendo iteraciones innecesarias y el tiempo de cómputo sin afectar a la calidad final del modelo.

Bibliografía

[1] Plain English. The gradient descent family explained: Batch, mini-batch and stochastic, 2021. URL https://python.plainenglish.io/the-gradient-descent-family-explained-batch-mini-batch-and-stochastic-89da9904f43b. Accedido en febrero de 2026.

[2] IBM. £qué es el descenso de gradiente estocástico?, 2026. URL https://www.ibm.com/mx-es/think/topics/stochastic-gradient-descent. Accedido en febrero de 2026.

[3] I. Belcic and C. Stryker. £qué es la tasa de aprendizaje en el machine learning?, 2026. URL https://www.ibm.com/mx-es/think/topics/learning-rate. Accedido en febrero de 2026.